# PAYSIM FRAUD INTELLIGENCE PLATFORM

## Technical Project Documentation & Design Pack

**Project Planning • Stakeholders • Data & Database Design • ML Methodology • UI/UX • Testing**

**SCOPE** This is a historical PaySim synthetic-data prototype for post-transaction fraud monitoring and decision support. It does not connect to a real bank, authorize payments, or perform a real hold / reversal.

**Prepared: 30 October 2026**

Submission documentation for the DEPI graduation project

# 1. Executive Summary

The PaySim Fraud Intelligence Platform turns a large synthetic financial transaction history into an explainable analytical workflow. It combines exploratory data analysis (EDA), feature engineering, a class-weighted SGDClassifier using logistic loss, validation-based threshold selection, a three-tier risk policy, SQL Server persistence, alert records, and a Dash / Plotly dashboard. The dashboard is intended to help an analyst inspect patterns and investigate historical transactions.

The model was trained on the chronological training period and evaluated on later validation and final-test periods. The final test contains 89,466 transactions and 1,252 fraud cases. At the fixed high-risk threshold selected on validation, the holdout produced 1,236 true positives, 16 false negatives, zero false positives, 100% precision, 98.722% recall, 99.3569% F1, and 98.9589% Average Precision using raw decision margins.

**INTERPRETATION** These metrics describe one chronological holdout from a synthetic dataset. They are not evidence of guaranteed performance on real bank traffic.

# 2. Project Planning and Scope

## 2.1 Problem statement

Financial fraud detection is an imbalanced classification and prioritization problem. Fraud is rare relative to normal transactions, and the cost of a missed fraud can differ from the cost of a false alert. A useful prototype must therefore measure precision and recall, choose thresholds transparently, and present traceable decisions rather than relying on accuracy alone.

## 2.2 Objectives

- Profile the PaySim source data and document fraud behavior by transaction type.
- Create leakage-aware chronological train, validation and test periods.
- Engineer amount and balance-behavior features and train a scalable linear classifier.
- Select a high-risk threshold on validation and freeze it before final-test evaluation.
- Convert raw model scores into LOW / MEDIUM / HIGH operational tiers.
- Persist validation and test transactions, predictions and alert records in SQL Server.
- Build a dashboard for KPIs, charts, transaction search, row inspection and alert filtering.

## 2.3 In-scope / out-of-scope

| In scope for this iteration | Not yet in scope |
|---|---|
| Historical PaySim analysis; model training/evaluation; experimental risk policy; SQL Server evaluation store; local Dash dashboard; simulated alert / hold fields. | Live bank integration; pre-authorization blocking; real payment execution; production case management; regulatory certification; live threat detection; production-grade cloud DB migration. |

### 2.4 Delivery phases

| Phase | Deliverable | Status evidenced in this iteration |
|---|---|---|
| 1. Data & EDA | Source profiling, fraud behavior findings | Completed |
| 2. ML | Chronological split, engineered Parquet, SGD model, validation/test reports | Completed |
| 3. Risk & alerts | Risk policy, risk engine, JSONL alert routing | Completed |
| 4. SQL persistence | Transactions, FraudPredictions, FraudAlerts and integrity checks | Completed |
| 5. Dashboard | Dash layout, CSS, Plotly charts, four callbacks, local server | Local dashboard running and integration passed |
| 6. Security labs | SQL Injection defensive lab and local MITM integrity simulation | Planned follow-on modules |
| 7. Cloud publication | Git repository + remote data endpoint + cloud service | Not yet deployed |

# 3. Stakeholder Analysis

| Stakeholder | Need / concern | Project response |
|---|---|---|
| Project owner / developer | Reproducibility, learning, demonstrable end-to-end flow | Python scripts, model bundle, SQL schema/load scripts, dashboard, reports and evidence. |
| Evaluator / instructor | Clear scope, methodology, proof of work and limitations | Structured presentation, technical documentation, test evidence and honest claims. |
| Fraud analyst (intended user) | Prioritized cases, filters, traceable features and alert state | Risk tiers, transaction explorer, raw decision score, action label and alert table. |
| ML engineer | Leakage prevention, class imbalance, validation discipline and drift awareness | Chronological split, feature exclusion, class weighting, raw-margin analysis and separate test. |
| Database administrator | Consistency, keys, referential integrity and query safety | Primary/foreign keys, unique SourceRowNumber, indexes and parameterized filters. |
| Security reviewer | Credential protection, SQL injection resistance, integrity and clear boundaries | Parameterized SQL in the current data access layer; separate planned SQLi/MITM labs. |

# 4. Dataset and EDA

### 4.1 Source dataset

The source file `PS_20174392719_1491204439457_log.csv` contains 6,362,620 rows and 11 columns: step, type, amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud, and isFlaggedFraud. It is synthetic PaySim data; account identifiers are synthetic transaction identifiers, not real customer identities.

| Profile | Observed value |
|---|---|
| Total rows | 6,362,620 |
| Fraud rows | 8,213 |
| Fraud prevalence | 0.1291% approximately |

| Profile | Observed value |
|---|---|
| | |
| Original columns | 11 |
| Missing / invalid / negative checks | No missing values, invalid numeric values or negative values were reported in the source audit. |
| Fraud transaction types | TRANSFER 4,097; CASH_OUT 4,116; all other types 0 fraud labels in this file. |

### 4.2 EDA findings by transaction type

| Observed relationship | TRANSFER | CASH_OUT | Interpretation |
|---|---|---|---|
| Mean amount: normal vs fraud | 906,229.01 vs 1,480,891.67 | 173,917.16 vs 1,455,102.59 | Fraud mean amount is higher in both observed fraud-bearing types; amount is not a standalone rule. |
| Origin balance emptied: normal vs fraud | 42.50% vs 96.02% | 42.78% vs 99.08% | Strong association in these two types. |
| Origin balance mismatch: normal vs fraud | 96.21% vs 0.51% | 89.16% vs 0.58% | Counter-intuitive; mismatch is not a universal fraud signal. |
| Destination balance mismatch: normal vs fraud | 22.07% vs 99.95% | 22.70% vs 13.24% | Strongly type-dependent; especially distinctive for TRANSFER in this dataset. |

### 4.3 Hypotheses

- H1: Fraud-bearing transactions have higher amounts. Observed as an association in TRANSFER and CASH_OUT.
- H2: Fraud is associated with the origin account being emptied. Strongly supported in the observed groups.
- H3: Origin balance mismatch is a fraud marker. Not supported as a universal rule; normal transactions show a much higher mismatch rate in this analysis.
- H4: Destination balance mismatch is associated with fraud. Strong for TRANSFER, not for CASH_OUT; interpretation must be conditioned on transaction type.

EDA generates hypotheses; it does not establish causality. These findings are specific to the PaySim synthetic data and its transaction-generation rules.

## 5. Temporal Experimental Design

Rows were split by PaySim step to preserve chronological order. This is more realistic than random splitting when transaction distributions may change over time.

| Period | Step range | Rows | Fraud | Fraud rate |
|---|---|---|---|---|
| Train | 1–520 | 6,082,007 | 5,781 | 0.0951% |
| Validation | 521–631 | 191,147 | 1,180 | 0.6173% |
| Final test | 632–743 | 89,466 | 1,252 | 1.3994% |

Fraud prevalence rises across the timeline. This is evidence of temporal distribution shift and a warning that performance may change with time. Validation is used for model diagnostics and threshold selection. The final test remains separate and is evaluated only after the high threshold has been selected.

# 6. Feature Engineering and Leakage Controls

The prepared Parquet feature table contains 6,362,620 rows, 13 columns and preserves all 8,213 fraud labels. Engineered features include `log_amount`, `orig_balance_emptied`, `orig_balance_error`, `dest_balance_error`, `orig_balance_mismatch` and `dest_balance_mismatch`, alongside step, transaction type, amount, identifiers, target and an excluded flag field.

| Feature / field | Role | Why |
|---|---|---|
| step | Model feature | Transaction time index in PaySim. |
| type | Model feature after encoding | Transaction category; behavior differs by type. |
| amount / log_amount | Model features | Raw amount and compressed scale for skewed values. |
| Balance-derived features | Model features | Post-transaction context and consistency / emptied-balance signals. |
| nameOrig / nameDest | Identifiers only | Retained for traceability/search; excluded from predictive X. |
| isFraud | Target only | Never included as a predictor. |
| isFlaggedFraud | Excluded from model | Avoid reliance on an auxiliary system flag that would distort the independent model assessment. |

**TIMING LIMITATION** Features derived from `newbalanceOrig` and `newbalanceDest` are only known after transaction processing. The prototype is therefore post-transaction monitoring. A pre-authorization model requires features available before approval.

# 7. Machine Learning Methodology

## 7.1 Model selection

The project uses scikit-learn `SGDClassifier(loss="log_loss")`. This is a linear classifier trained with stochastic gradient descent and logistic loss. It is not a Random Forest, neural network or the `LogisticRegression` estimator, although its log-loss objective is logistic-regression-like.

The workflow includes numerical feature scaling with `StandardScaler`, categorical handling for transaction type, class weighting for imbalance, and incremental `partial_fit` training over batches. The fitted preprocessing and classifier are saved together in a model bundle.

## 7.2 Mathematical explanation

For a feature vector x, the model calculates a linear decision margin: $z = w^T x + b$. The sigmoid converts the margin to $p = 1 / (1 + \exp(-z))$. Logistic loss for one labeled row is $L = -[y \log(p) + (1-y) \log(1-p)]$. SGD updates parameters in the direction that reduces the loss: $w \leftarrow w - \eta \nabla L(w)$, where $\eta$ is the learning rate. Regularization discourages unnecessarily large weights and can improve generalization.

### 7.3 Why scaling matters

Features such as transaction amount may have a much larger numeric range than binary or ratio-like engineered features. StandardScaler puts numerical features on comparable scale using training-set statistics. The scaler must be fitted on training data only, then reused unchanged for validation and test; otherwise information from future periods can leak into preprocessing.

### 7.4 Class weights

With a fraud prevalence near 0.129%, unweighted optimization can be dominated by the normal class. Class weighting increases the loss contribution of the minority class. A common balanced reference is $w_i = N / (K \times n_i)$, with N training rows, K classes and $n_i$ rows in class i. The project applied class weighting; do not state exact numeric weights unless they are verified from the training script or saved configuration. The formula explains the common principle, not necessarily the exact configured dictionary.

## 8. Probability Saturation and Raw Decision Margins

The initial probability output became numerically saturated near 1.0 for many rows. This happens because the sigmoid rapidly approaches 1 as z becomes large and positive. In floating-point calculations, many distinct large positive margins can map to exactly 1.0, creating ties. Average Precision depends on ranking; ties can destroy ranking detail even when the underlying margins remain separable.

| Validation diagnostic | Observed result |
|---|---|
| Average Precision using saturated probability output | 0.2046 |
| Average Precision using raw decision_function margin | 0.995086 |
| Selected high threshold on raw validation margin | 213.80866687194845 |
| Validation at selected threshold | Precision 1.0000; recall 0.994068; F1 0.997025; TP 1,173; FN 7; FP 0 |

**CRITICAL LABELING** `decision_function` returns an uncalibrated margin, not a probability. The dashboard correctly labels it `Raw Decision Score`. Do not present 213.8 as 213.8% or call it a 99% probability.

## 9. Risk Policy and Alert Logic

The risk engine applies two validation-derived cutoffs to the raw decision margin: medium threshold 54.304175371966295 and high threshold 213.80866687194845.

| Risk tier | Condition | Recommended action |
|---|---|---|
| LOW | score < 54.304175371966295 | MONITOR |
| MEDIUM | 54.304175371966295 ≤ score < 213.80866687194845 | MANUAL_REVIEW |
| HIGH | score ≥ 213.80866687194845 | ALERT_AND_SIMULATED_HOLD |

The medium cutoff is the 99th percentile of normal validation raw margins. It is an experimental tail-control threshold, not an approved bank policy or calibrated probability. The high threshold was selected to maximize F1 on validation. Both are stored in the risk policy artifact and applied to later records.

The Alert System routes HIGH records to a high-risk alert log and MEDIUM records to a manual-review queue. These are local / SQL project records; there are no real notifications, payment holds or bank-side actions.

# 10. Model Evaluation

## 10.1 Validation

| Metric | Validation result |
|---|---|
| Precision | 1.0000 |
| Recall | 0.994068 |
| F1 | 0.997025 |
| Average Precision (raw margins) | 0.995086 |
| Confusion matrix TN / FP / FN / TP | 185,428 / 0 / 7 / 1,173 |

## 10.2 Final chronological test

| Metric | Final-test result |
|---|---|
| Rows / fraud | 89,466 / 1,252 |
| Accuracy | 0.999821 (99.9821%) |
| Precision | 1.000000 (100%) |
| Recall | 0.987220 (98.7220%) |
| F1 | 0.993569 (99.3569%) |
| Average Precision (raw margins) | 0.989589 |
| Confusion matrix TN / FP / FN / TP | 88,214 / 0 / 16 / 1,236 |
| Fraud prevalence | 1.399414% |
| Precision lift | 71.46× |

The threshold was selected on validation and applied unchanged to the final test. The test has zero false positives among 88,214 normal transactions in this sample and misses 16 of 1,252 fraud cases. A zero false-positive count in one sample does not guarantee zero false positives in production.

# 11. SQL Server Database Design

Database: `PaySimFraudDB` on the local SQL Server Developer instance. The evaluation loader inserts only validation and final-test records, not the 6.08 million training rows. This keeps the dashboard evaluation store smaller and directly tied to held-out assessments.

| Table | Purpose | Key fields / relationships |
|---|---|---|
| dbo.Transactions | Source transaction facts and target label for evaluation. | TransactionID primary key; SourceRowNumber unique; step, type, amount, identifiers, balance fields, isFraud. |
| dbo.FraudPredictions | One model output per loaded transaction. | PredictionID primary key; TransactionID foreign key; DatasetSplit; RawDecisionScore; PredictedFraud; RiskLevel; RecommendedAction; model name/version. |
| dbo.FraudAlerts | Investigation / alert records for high and medium decisions. | AlertID primary key; links to PredictionID and TransactionID; |

| Table | Purpose | Key fields / relationships |
|---|---|---|
| | | AlertType, AlertStatus, message, timestamps and SimulatedHold. |

| Integrity check | Observed result |
|---|---|
| Transactions | 280,613 |
| FraudPredictions | 280,613 |
| FraudAlerts | 5,255 |
| Duplicate SourceRowNumber | 0 |
| Orphan predictions | 0 |
| Orphan alerts | 0 |

The Python data-access layer uses pyodbc, parameterized query values and bounded result limits. The local connection currently relies on Windows Authentication. Cloud deployment requires a remote SQL endpoint and an authentication approach supported by the hosting environment; local Windows authentication cannot be reused by a Linux cloud container.

# 12. Dashboard UI/UX and Operation

The dashboard is built with Dash, Plotly and Dash Bootstrap Components. The visual system uses a dark Midnight FinTech theme and a responsive layout. The layout includes a brand header, global filters, six KPI cards, three analytics charts, an Alert Center, a Transaction Explorer, selected-transaction details, database status, error display and last-refresh timestamp.

| Region | What the user sees | Purpose |
|---|---|---|
| Global filters | split-filter, type-filter, risk-filter | Choose evaluation period, transaction type and risk tier. |
| KPIs | Total transactions, actual fraud, prevalence, high risk, manual review, F1 | Executive summary of selected period/type. The KPI query does not accept risk_level. |
| Type chart | Actual fraud and predicted fraud bars; transaction-volume line on secondary axis | Compare fraud and volume across transaction categories. |
| Risk donut | LOW / MEDIUM / HIGH transaction counts | Understand how transactions are distributed across experimental risk tiers. |
| Temporal chart | Transaction-volume bars; actual/predicted fraud lines; 7-step moving average | Observe changes over PaySim time steps. |
| Transaction Explorer | Search + 14-column table + selected-row details | Find by source row / sender / receiver and inspect model / risk outputs. |
| Alert Center | 15-column table with status and category filters | Review persisted HIGH_RISK and MANUAL_REVIEW records. |
| Refresh / health | Manual refresh, 30-second interval, connection status and timestamp | Keep dashboard output current relative to the connected database. |

## 12.1 Callback architecture

Four callbacks are registered: (1) analytics updates the six KPIs, three figures, database status, error and last-updated text; (2) transaction table refreshes the bounded transaction rows; (3) selected-row callback

creates the transaction detail panel; (4) alert table updates the alert records. Inputs include dropdown values, the manual refresh button and `dcc.Interval` every 30 seconds. The explorer also listens to the debounced search input.

### 12.2 Exact filter semantics

Dataset split is passed to all data views. Transaction type is passed to KPIs, risk distribution, temporal trend, transaction explorer and alert query. Risk level is passed to the type-analysis query and transaction explorer; it is not a parameter of `fetch_kpis`, `fetch_risk_distribution` or `fetch_temporal_trend` in the current version. This is an intentional statement of current implementation behavior, not a claim that the risk filter affects every chart.

### 12.3 Current limits

The transaction and alert tables request up to 100 rows per refresh (data access permits up to 500). Browser-side sorting/pagination applies to the fetched subset, not the entire SQL table. This is a bounded first version, not server-side pagination over all 280,613 rows.

## 13. Verification and Test Evidence

| Test / check | Evidence result |
|---|---|
| Feature Parquet integrity | 6,362,620 rows; 8,213 fraud labels; no nulls. |
| Risk-engine smoke test | One LOW, one MEDIUM and one HIGH sample returned the expected action tier. |
| Alert-system smoke test | LOW monitored; MEDIUM manual-review queue; HIGH alert JSONL routing. |
| SQL loader and integrity | 280,613 transactions and predictions; 5,255 alerts; no duplicate source-row mapping or orphans. |
| Plotly chart tests | Type: 3 traces × 5 points; Risk: 1 pie trace × 3 slices; Temporal: 4 traces × 223 points. |
| Analytics callback execution | 12 outputs populated; database connected to PaySimFraudDB; dashboard error None. |
| Transaction explorer execution | 100 rows returned; 14 columns; selected transaction details rendered. |
| Alert Center execution | 100 alerts returned; 15 columns; HIGH_RISK filter returned only HIGH_RISK rows. |
| Full Dash integration | 4 registered callbacks; 23 layout IDs; 0 missing IDs; 0 duplicate outputs; CSS asset exists. |
| Local server | Dash launched at http://127.0.0.1:8050; HTTP 200 responses observed for app, assets and Dash update requests. |

# 14. Security and Future Work

## 14.1 SQL Injection defensive lab - planned

A separate controlled lab will demonstrate the difference between unsafe string-concatenated SQL and parameterized queries, use harmless local test inputs, implement validation and least privilege, and verify that malicious input is treated as data. The existing dashboard data-access methods use parameter placeholders for search and filter values; the dedicated lab will provide explicit tests and educational evidence.

## 14.2 Man-in-the-Middle integrity simulation - planned

A separate local-only simulation will model a client, an intermediary that alters a message, and a server that verifies a message authentication code or digital signature. A valid integrity tag should pass; a modified payload should fail verification and generate a security event. This will not intercept real network traffic.

## 14.3 Further engineering work

- Test on real, properly governed data with a realistic deployment-time feature contract.
- Evaluate calibration and choose thresholds from explicit false-positive / false-negative costs and human-review capacity.
- Monitor data drift, label delay, subgroup behavior and performance over time.
- Add audit-user identity, role-based access, model/policy versioning, alert resolution workflow and server-side pagination.
- Deploy through GitHub and a cloud web service only after a reachable, secured cloud database or a deliberately read-only demo data layer exists.

# 15. Reproducibility and Deployment Notes

The local application is started from the project root with `python -m src.dashboard.app`. It serves locally at `http://127.0.0.1:8050`. A cloud deployment is not the same as a local run: a hosted Linux process cannot see `D:\` paths or authenticate to the developer workstation using Windows Authentication. The cloud version must use environment-based connection settings and a remote database endpoint reachable from the host. Secrets must be configured in the host dashboard and must never be committed to Git.

Render is one possible host for a Dash web service. Its official documentation describes Python web-service deployment and notes that free services spin down after 15 minutes without inbound traffic. Free web services also have an ephemeral filesystem, so local writes are not durable. For an evaluation demo, this can be acceptable if the data is read from a persistent remote database and the cold-start behavior is disclosed.

# Appendix A. Key numerical facts

| Item | Value |
|---|---|
| Raw rows / features | 6,362,620 / 11 |
| Fraud population | 8,213 (0.1291%) |

| Item | Value |
|---|---|
| Feature artifact | 6,362,620 rows × 13 columns |
| Training / validation / test rows | 6,082,007 / 191,147 / 89,466 |
| Medium / high raw thresholds | 54.304175371966295 / 213.80866687194845 |
| Final-test TP / FP / FN / TN | 1,236 / 0 / 16 / 88,214 |
| SQL evaluation rows / predictions / alerts | 280,613 / 280,613 / 5,255 |
| Dashboard callbacks / components | 4 / 23 |

# 16. Dataset-specific balance-field warning

The PaySim Kaggle dataset metadata explicitly warns that transactions detected as fraud are cancelled and states that oldbalanceOrg, newbalanceOrig, oldbalanceDest and newbalanceDest should not be used for fraud detection. The current experimental feature set includes balance-derived features. This makes the current model a useful engineering and diagnostic prototype, but it raises a serious dataset-specific validity concern: the balance variables may encode simulator cancellation behavior or post-event information rather than a signal available in a deployable fraud decision. The strong test metrics must therefore be reported with this caveat.

Required next experiment: train a second model after removing all four raw balance fields and every derived balance feature (orig_balance_emptied, orig_balance_error, dest_balance_error, orig_balance_mismatch and dest_balance_mismatch). Keep the exact same chronological split, fit scaler and class weights on training only, select the threshold on validation only, freeze it, and evaluate once on the final test. Compare precision, recall, F1, Average Precision, confusion matrix, and stability across the three time periods. This is an ablation study and should be reported alongside, not silently substituted for, the current run.

# 17. References


- E. A. López-Rojas, A. Elmir, and S. Axelsson. “PaySim: A financial mobile money simulator for fraud detection.” 28th European Modeling and Simulation Symposium (EMSS), 2016. https://www.msc-les.org/proceedings/emss/emss2016/emss2016_249.html
- Kaggle. “Synthetic Financial Datasets For Fraud Detection (PaySim1),” ealaxi. Dataset metadata, including field definitions, licence and balance-field warning. https://www.kaggle.com/datasets/ealaxi/paysim1/metadata
- scikit-learn Developers. “SGDClassifier” API documentation, version 1.9.1. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html
- scikit-learn Developers. “Stochastic Gradient Descent” user guide. https://scikit-learn.org/stable/modules/sgd.html
- Render. “Deploy a Flask App” and “Deploy for Free” documentation. https://render.com/docs/deploy-flask ; https://render.com/docs/free